Stretch 2 Component Test Runbook

Pill bottle project · bottle_tracking package · T0, T1a, T1b, T2, T3, T4, T6, T8, T9, T10, T11

> Run everything on the robot over `ssh -Y hello-robot@192.168.1.172`. Type commands one per line. Don't add `#` comments on the same line in your Mac's terminal (zsh treats them as part of the command).

auto tester logs results by itself manual log each trial with run_manual_test.py

One time only

1. YOLO and your model

ultralytics is already installed on the robot. Your model (`/home/hello-robot/pill_bottle_robot/final_pill_bottle.pt`, classes pill_bottle, cap, label) is set in params.yaml as `model_path`, so you don't need to type it. If you ever reinstall, use `pip3 install ultralytics "numpy<2"`, because NumPy 2 breaks ROS 2 Humble.

To start the new tests from a clean slate, move the old results aside once:

```
mv ~/stretch_test_results ~/stretch_test_results_old_v1
```

2. Rebuild after pulling the update

```
cd ~/stretch2pillbottlesort
git fetch
git reset --hard origin/main
cd ~/ament_ws
colcon build --packages-select bottle_tracking --symlink-install
source install/setup.bash
```

3. Check the detector sees the bottle (upright image, green box)

Start the session terminals below first, then run:

```
ros2 run rqt_image_view rqt_image_view /pill_bottle/debug_image
```

If the image is sideways, set `rotate_image` in `ros2_ws/src/bottle_tracking/config/params.yaml` to `cw`, `ccw` or `none` (whichever matches your training images), then restart terminal 3. A red box means it sees the bottle but has no depth there.

4. Calibrate the arm (before T1a and T1b)

The arm and wrist will move, so clear the robot's right side first.

```
ros2 run bottle_tracking calibrate_arm --ros-args -p write:=true
```

Then put your pick code's gripper values into `gripper_open` and `gripper_closed` in params.yaml.

Every session: setup

If the robot was powered off, home it first, with the driver stopped:

```
stretch_robot_home.py
stretch_robot_stow.py
```

Open 4 terminals and run `source ~/ament_ws/install/setup.bash` in each one first.

Terminal	Command (leave running)
1	`ros2 launch stretch_core stretch_driver.launch.py broadcast_odom_tf:=True mode:=navigation`
2	`ros2 launch stretch_core d435i_high_resolution.launch.py`
3	`ros2 launch bottle_tracking bottle_tracking.launch.py`
4	`python3 ~/stretch2pillbottlesort/testing/watch_bottle.py` (prints SEES / LOST)

In a 5th terminal for the tests:

```
cd ~/stretch2pillbottlesort
P=$(ros2 pkg prefix bottle_tracking)/share/bottle_tracking/config/params.yaml
```

Every tester pauses between trials so you can move the bottle. Press Enter to skip the wait, or change it with `-p pause_s:=30`.

Tests

T0: Logging setup

```
python3 testing/summarize_results.py
```

Paste the output into the T0 issue and close it.

T1a: Find the bottle and print the plan (head only) auto

Only the head moves; the base never turns, not even to search. Each trial prints **PLAN** and the base and arm plan, or **NO PLAN**, the reason, and what would make a new plan possible. It doesn't ask you anything. At the end it saves a picture of every planned approach (camera view and top-down drawing) plus one overview of all trials.

```
ros2 run bottle_tracking t1a_plan_tester --ros-args --params-file $P -p trials:=3
```

T1b: Find, plan, drive and grab auto

Same as T1a, then it carries out the plan. Before driving, it pulls the arm in and raises the lift above table height. The bottle's location is saved, so losing sight of it while turning is fine. After driving it looks at the saved spot for up to 20 s. If it doesn't see the bottle, it searches the area around the saved spot with the head; if there's still nothing, it assumes nothing changed and uses the saved location. If the arm can't reach yet, it turns or drives again (up to 2 more times) to put the arm in front of the bottle. Then it grabs (Enter to confirm), lifts and holds for 5 s, you say what happened, and it puts the bottle back. Run `calibrate_arm` first and clear the path.

```
ros2 run bottle_tracking t1b_execute_tester --ros-args --params-file $P -p trials:=3
```

T2: Find the bottle and say where it is auto

Uses the head only (the base never moves). Reports whether it sees the bottle, its position in the fixed frame, its distance and direction, and whether the robot needs to move to grab it. 20 s pause between trials. Saves a picture for each trial.

```
ros2 run bottle_tracking t2_find_tester --ros-args --params-file $P -p trials:=5
```

T3: Watch while moving, then remember auto

Part A: moves around with head tracking on (turns, drives, a mix). Part B runs right after: it finds the bottle first (turning the base if needed), then moves with the head frozen and looks back at the remembered spot. Leave about 1 m of clear space.

```
ros2 run bottle_tracking t3_tracking_tester --ros-args --params-file $P -p trials:=3
```

To run only one part: `-p part:=A` or `-p part:=B`

T4: "Nothing there" recovery manual

Let the robot find the bottle, then take the bottle away. The watcher should print LOST, and this should show `MISSING` within about 3 s:

```
ros2 topic echo /bottle_memory/state --once
python3 testing/run_manual_test.py T4
```

T6: Distance at different spots, base still auto

60 s pause between spots. Prints the distance from the base and from the camera, and how steady the reading is. Type the tape-measured distance (or press Enter to skip) to get the error. Saves a picture for each spot.

```
ros2 run bottle_tracking t6_distance_tester --ros-args --params-file $P -p trials:=5
```

T8: Grasp only manual

Put the gripper around the bottle, close, lift and hold for 5 s:

```
python3 testing/run_manual_test.py T8
```

T9, T10, T11: Orientation, in-hand verify, retry manual

```
python3 testing/run_manual_test.py T9
python3 testing/run_manual_test.py T10
python3 testing/run_manual_test.py T11
```

Manual tester keys: `s` SUCCESS, `m` MISS, `n` NOT_FOUND, `d` DROP, `t` TIMEOUT, `f` FALSE_POSITIVE, `e` ERROR, `q` quit.

Pictures

T1a, T1b, T2 and T6 save the camera view for every trial in `~/stretch_test_results/images/`. A **green** box marks the bottle the robot is using (with confidence and distance). **Red** boxes are detections it ignored, labeled with why (for example "too far"). T1a and T1b pictures also show the top-down plan: start in gray, the dashed path, the planned end in green, and the arm in orange. Copy them to your Mac from a Mac terminal:

```
scp -r hello-robot@192.168.1.172:stretch_test_results/images ~/Desktop/stretch_images
```

End of every session

```
python3 testing/summarize_results.py --session latest
python3 testing/summarize_results.py --test T2 --markdown
```

Paste each test's markdown into its GitHub issue.

If something doesn't work

Problem	Fix
Testers never find the bottle	Look at the trial picture and the "detections received" count it prints. If the count is 0, check terminal 3 for detector errors. If there's no box on the bottle, lower `conf_threshold` or fix `rotate_image` in params.yaml.
Bottle position jumps far away	A look-alike is being picked up. Detections over `max_detect_depth_m` (3 m) or above `max_bottle_height_m` (1.4 m) are ignored; lower them if needed.
Head turns away from the bottle	Set `pan_sign: -1.0` (or `tilt_sign`) in params.yaml and restart terminal 3.
Arm stops short or goes too far	Run `calibrate_arm` again.
Gripper doesn't close enough	Lower `gripper_closed` (check with T8).
Distances are always off by the same amount	Change `depth_offset_m` by the average signed error from T6.